

NP-hard recargado

(Ideas para problemas constructivos y de codificación)

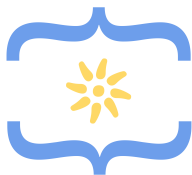
Agustín Santiago Gutiérrez

Training Camp 2026



Gracias Sponsors!

Organiza



Facultad de
INFORMÁTICA



UNIVERSIDAD
NACIONAL
DE LA PLATA

Diamond Sponsor



- 1 Inicia nuestra aventura con algo que nada que ver
- 2 Algunos ejemplos con backtracking y otras opciones
- 3 Reducciones para demostrar que los problemas son NP-hard
 - Clases de complejidad
 - Bibliografía

- 1 Inicia nuestra aventura con algo que nada que ver
- 2 Algunos ejemplos con backtracking y otras opciones
- 3 Reducciones para demostrar que los problemas son NP-hard
 - Clases de complejidad
 - Bibliografía

¿Qué complejidad tiene esto?

```
procedure fibonacci( $n$ )  
  if  $n < 2$  then return 1  
  else return  $f(n - 1) + f(n - 2)$ 
```

¿Qué complejidad tiene esto?

```
procedure fibonacci( $n$ )  
  if  $n < 2$  then return 1  
  else return  $f(n - 1) + f(n - 2)$ 
```

- $O(\phi^n)$, donde $\phi = \frac{1+\sqrt{5}}{2}$. Aproximadamente $O(1.618^n) = O(2^{0.694n})$
- ¿Por qué?

Recurrencias lineales

- Calcular tiempos de recursiones, puede requerir recurrencias
- Si $T(n)$ es el tiempo que demora la función anterior:
- $T(n) = T(n-1) + T(n-2)$
- ¿Y cómo es la fórmula matemática de una recurrencia lineal?

En realidad sería $T(n) = T(n-1) + T(n-2) + C$ para alguna constante C que acote el cómputo realizado por las instrucciones de propia $f(n)$ además del tiempo que consumieron las ejecuciones de $f(n-1)$ y $f(n-2)$.
Si rehacemos todas las cuentas de esta charla con cuidado poniendo los correspondientes C ahí, no cambia ninguna conclusión. Así que los voy a ignorar, y si quieren asegurarse de que es cierto que no cambia nada, les queda de ejercicio.

Truquito: λ^n

- Probamos “porque sí” a ver si funciona $f(n) = \lambda^n$
- $T(n) = T(n-1) + T(n-2)$
- $\lambda^n = \lambda^{n-1} + \lambda^{n-2}$
- $\lambda^n - \lambda^{n-1} - \lambda^{n-2} = 0$
- $\lambda^2 - \lambda^1 - \lambda^0 = 0$
- $P(\lambda) = 0$ donde $P(X) = X^2 - X - 1$
- P se llama polinomio característico de la recurrencia
- En este caso las raíces son $\lambda = \frac{1 \pm \sqrt{5}}{2}$
- Para cualquier A, B vale que $f(n) = A \left(\frac{1 - \sqrt{5}}{2} \right)^n + B \left(\frac{1 + \sqrt{5}}{2} \right)^n$ cumple

Caso raíz múltiple

- Si λ es raíz de multiplicidad k del característico P , entonces $f(n) = n^i \lambda^n$ es solución de la recurrencia, $0 \leq i < k$ con i entero
- TODAS las soluciones que existen son combinación lineal de estas d soluciones “básicas”, donde d es el grado de P
- Esto incluye las raíces complejas, que pueden ser necesarias en la fórmula incluso si todos nuestros valores son reales.

Resolución asintótica de recurrencia lineal

- Si tenemos una recurrencia lineal con coeficientes $a_i \in \mathbb{C}$:
- $f(n) = a_1 f(n-1) + a_2 f(n-2) + \dots + a_d f(n-d)$
- El característico es $P(X) = X^d - a_1 X^{d-1} - a_2 X^{d-2} - \dots - a_d X^0$
- Si $\lambda \in \mathbb{C}$ es una raíz de P de máximo valor absoluto, y de entre ellas una con máxima multiplicidad k , entonces $|f(n)| = O(n^{k-1} |\lambda|^n)$

Ejemplo

- $f(n) = 2f(n-1) - 9f(n-2) + 18f(n-3)$
- $P(X) = X^3 - 2X^2 + 9X - 18 = (X + 3i)(X - 3i)(X - 2)$
- Entonces sabemos que $|f(n)| = O(3^n)$
- Empíricamente con casos base $f(0) = f(1) = f(2) = 1$:

n	$f(n)$	$\frac{ f(n) }{3^n}$
0	1	1
1	1	0,33
2	1	0.11
3	11	0.41
4	31	0.38
5	-19	0.08
6	-119	0.16
7	491	0.22
8	1711	0.26
9	-3139	0.16
10	-12839	0.22

Complejidad de función recursiva no es cualquier cosa

- Si tenemos una función recursiva, que por las llamadas que hace da una recursión de tiempos como la anterior:
- $T(n) = a_1 T(n-1) + a_2 T(n-2) + \dots + a_d T(n-d)$
- El característico es $P(X) = X^d - a_1 X^{d-1} - a_2 X^{d-2} - \dots - a_d X^0$
- En esta aplicación particular son $a_i \geq 0$ enteros, al menos uno positivo
- Teorema: P tiene
 - Exactamente una raíz real positiva, que llamamos ρ
 - ρ es una raíz de máximo valor absoluto de P
 - Todas las raíces de máximo valor absoluto de P son raíces simples
- Demostración:
 - Regla de los signos de Descartes:
https://en.wikipedia.org/wiki/Descartes%27_rule_of_signs
 - Teorema de Perron-Frobenius: https://en.wikipedia.org/wiki/Perron%E2%80%93Frobenius_theorem
 - Un par de cuentitas y observaciones

Corolario: La receta para sacar la complejidad

- Si tenemos una función recursiva, que por las llamadas que hace da una recursión de tiempos como la anterior:
- $T(n) = a_1 T(n-1) + a_2 T(n-2) + \dots + a_d T(n-d)$
- El característico es $P(X) = X^d - a_1 X^{d-1} - a_2 X^{d-2} - \dots - a_d X^0$
- Sea $\rho \in \mathbb{R}_{>0}$ la única raíz real positiva de P
- $T(n) = O(\rho^n) = O(2^{(\lg \rho)n})$

¿Y esto otro qué complejidad?

```
procedure fibonacci( $n$ )  
  if  $n < 2$  then return 1  
  else  
    if  $n \bmod 7 \in \{2, 3, 5\}$  then return  $f(n - 1) + f(n - 2)$   
    else  
      if  $\text{hashSuperLoco}(n) = 1$  then return  $2f(n - 1) + f(n - 3)$   
      else return  $4f(n - 2) + 3f(n - 4)$ 
```

¿Y esto otro qué complejidad?

```
procedure fibonacc( $n$ )  
  if  $n < 2$  then return 1  
  else  
    if  $n \bmod 7 \in \{2, 3, 5\}$  then return  $f(n - 1) + f(n - 2)$   
    else  
      if  $\text{hashSuperLoco}(n) = 1$  then return  $2f(n - 1) + f(n - 3)$   
      else return  $4f(n - 2) + 3f(n - 4)$ 
```

- Es una recursión lineal “mezcla”: siempre aplica un paso de recursión lineal, pero cuál aplica depende en forma impredecible de n .
- ¿Cómo acotamos la complejidad?

La gran conjetura platense

- En un caso así, alcanza con:
 - Calcular cada característico y su correspondiente ρ_i
 - Tomar $\rho = \max_i(\rho_i)$
 - La complejidad es $O(\rho^n)$
- ¿Está bien hacer esto así de simple para resolver recurrencias lineales, y de esa manera terminar sacando la complejidad?
- Voten: ¿Sí? ¿No?

La gran conjetura platense

- En un caso así, alcanza con:
 - Calcular cada característico y su correspondiente ρ_i
 - Tomar $\rho = \max_i(\rho_i)$
 - La complejidad es $O(\rho^n)$
- ¿Está bien hacer esto así de simple para resolver recurrencias lineales, y de esa manera terminar sacando la complejidad?
- Voten: ¿Sí? ¿No?
- ¡Depende! Todos pierden.

La gran conjetura platense es FALSA EN GENERAL

- Para recursiones lineales cualesquiera, no es cierto que podamos acotar de esa manera una secuencia de operaciones mezcla.
- Sea $f(n) = \begin{cases} -4f(n-1) - 4f(n-2) & \text{si } n \text{ par} \\ 4f(n-1) - 4f(n-2) & \text{si } n \text{ impar} \end{cases}$
- Característicos: $X^2 + 4X + 4 = (X + 2)^2$ y $X^2 - 4X + 4 = (X - 2)^2$
- Cada recursión en aislado crecería (en valor absoluto) como $n \cdot 2^n$
- La función combinada crece como 4^n (experimentarlo!)

La gran conjetura platense VALE EN ESTE USO

- Nuestras recursiones no son arbitrarias: $a_i \geq 0$, con alguno positivo
- Usando las propiedades anteriores y cuentitas se demuestra
- Las ideas clave:
 - Sobre los $x \in \mathbb{R}_{>0}$, $P(x)$ tiene una única raíz, así que es $P(x) < 0$ antes de la raíz y $P(x) > 0$ después.
 - Demostrar por inducción que el tiempo está acotado por $C \cdot \rho^n$
 - La monotonía anterior y la hipótesis inductiva permiten acotar que todos los casos pierden contra el de ρ

- 1 Inicia nuestra aventura con algo que nada que ver
- 2 Algunos ejemplos con backtracking y otras opciones
- 3 Reducciones para demostrar que los problemas son NP-hard
 - Clases de complejidad
 - Bibliografía

Máximo conjunto independiente

- Problema clásico en grafos, NP-hard
- Fuerza bruta: $O(n \cdot 2^n)$
- El problema H de ayer era resolverlo con $n = 40$. ¿Ideas?

Máximo conjunto independiente

- Problema clásico en grafos, NP-hard
- Fuerza bruta: $O(n \cdot 2^n)$
- El problema H de ayer era resolverlo con $n = 40$. ¿Ideas?
- Meet in the middle $O(2^{\frac{n}{2}})$

Máximo conjunto independiente

- Problema clásico en grafos, NP-hard
- Fuerza bruta: $O(n \cdot 2^n)$
- El problema H de ayer era resolverlo con $n = 40$. ¿Ideas?
- Meet in the middle $O(2^{\frac{n}{2}})$
- Backtracking básico + Memoization: $O(2^{\frac{n}{2}})$ “automáticamente” !!

Máximo conjunto independiente: más backtracking

- Básico:
 - $O(2^n)$ porque $T(n) = 2T(n-1)$

Máximo conjunto independiente: más backtracking

- Básico:
 - $O(2^n)$ porque $T(n) = 2T(n-1)$
- “Si es aislado lo elijo”: $\deg = 0$ es $T(n) = T(n-1)$
 - Y si $\deg \geq 1$ es $T(n) = T(n-1) + T(n-2)$
 - $\rho = 1$ y $\rho = \phi$ respectivamente
 - Queda $O(2^{0.694n})$

Máximo conjunto independiente: más backtracking

- Básico:
 - $O(2^n)$ porque $T(n) = 2T(n-1)$
- “Si es aislado lo elijo”: $\deg = 0$ es $T(n) = T(n-1)$
 - Y si $\deg \geq 1$ es $T(n) = T(n-1) + T(n-2)$
 - $\rho = 1$ y $\rho = \phi$ respectivamente
 - Queda $O(2^{0.694n})$
- “Si tiene grado 1 lo elijo”: $\deg = 1$ es $T(n) = T(n-2)$
 - Y si $\deg \geq 2$ es $T(n) = T(n-1) + T(n-3)$
 - $\rho = 1$ (aislados o grado 1) y $\rho = 1.466$ en otro caso
 - Queda $O(2^{0.551n})$

Máximo conjunto independiente: más backtracking

- Básico:
 - $O(2^n)$ porque $T(n) = 2T(n-1)$
- “Si es aislado lo elijo”: $\deg = 0$ es $T(n) = T(n-1)$
 - Y si $\deg \geq 1$ es $T(n) = T(n-1) + T(n-2)$
 - $\rho = 1$ y $\rho = \phi$ respectivamente
 - Queda $O(2^{0.694n})$
- “Si tiene grado 1 lo elijo”: $\deg = 1$ es $T(n) = T(n-2)$
 - Y si $\deg \geq 2$ es $T(n) = T(n-1) + T(n-3)$
 - $\rho = 1$ (aislados o grado 1) y $\rho = 1.466$ en otro caso
 - Queda $O(2^{0.551n})$
- “Si tiene grado 2, o lo uso, o uso los dos vecinos”:
 - $\deg = 2$ es $T(n) = 2T(n-3)$
 - Y si $\deg \geq 3$ es $T(n) = T(n-1) + T(n-4)$
 - $\rho = 1$ ($\deg \leq 1$), $\rho = \sqrt[3]{2} = 1.26$ ($\deg = 2$) y $\rho = 1.38$ ($\deg \geq 3$)
 - Queda $O(2^{0.465n}) = o(2^{\frac{n}{2}})$

Problema repintar

- ¿Lo pensaron?
`https://juez.oia.unsam.edu.ar/tasks/repintar`

Problema repintar

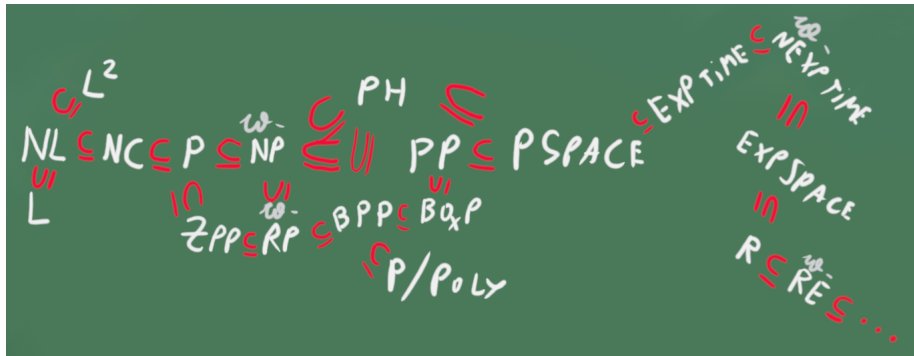
- ¿Lo pensaron?
<https://juez.oia.unsam.edu.ar/tasks/repintar>
- ¿Cómo hago una buena solución?

Problema repintar

- ¿Lo pensaron?
<https://juez.oia.unsam.edu.ar/tasks/repintar>
- ¿Cómo hago una buena solución?
- ¿Cómo podría acotar cuánto tarda sobre cualquier caso?

- 1 Inicia nuestra aventura con algo que nada que ver
- 2 Algunos ejemplos con backtracking y otras opciones
- 3 Reducciones para demostrar que los problemas son NP-hard
 - Clases de complejidad
 - Bibliografía

Clases de complejidad



Clases de complejidad (for dummies)

Si $P \neq NP...$

NP-HARD

P

?

Demostrar NP-hard, ¿Para qué?

- No perder tiempo tratando de resolver algo que no se puede
- Ganar intuición sobre qué cosas son P vs NP-hard
- Patrón super común: Caso general NP-hard, vs Caso específico P
- El mismo tipo de técnicas y trucos para demostrar que un problema es NP-hard, se utilizan mucho en problemas constructivos / interactivos / de encoding, o incluso para “traducir un problema a un código de notebook caja-negra” (flujo / tree reroot / segment tree lazy propagation / etc).

Teorema de Cook (1971)

- SAT es NP-hard.
- Permite demostrar algunos muy directos:
 - 3-SAT es NP-hard (via SAT)
 - Máximo conjunto independiente es NP-hard (via SAT)
 - Máximo cloque es NP-hard (via MCI)
 - Mínimo cubrimiento de aristas por vértices es NP-hard (via MCI)

- ¿Podemos mejorar $O(2^n)$ para SAT?
- ¿Y para 3-SAT?

- ¿Podemos mejorar $O(2^n)$ para SAT?
- ¿Y para 3-SAT?
- Para 3-SAT conseguimos $T(N) = 7T(N - 3)$, queda $O(2^{0.936n})$
- Lo mejor conocido en la teoría para 3-SAT es aproximadamente $O(2^{0.387n})$

Formas de pensar reducciones

- En su famoso libro, Garey y Johnson dan 3 “formas típicas de pensar” una reducción de un problema A a otro B
- Ni son exhaustivos, ni son excluyentes entre sí (hasta es subjetivo cuál se está usando)

Formas de pensar reducciones (cont.)

- Vamos a buscar una reducción polinomial desde A hasta B .
- B es el que queremos demostrar NP-hard, y A ya sabemos que es NP-hard.
- Restricción / Generalización [Cam. Hamilt. $\rightarrow$ Max Cam. Simple]
 - La más común
 - La más fácil
 - Para la que más claramente sirve conocer muchísimos NP-hard
 - Se pueden encontrar comenzando por B y fijando parámetros o restringiendo propiedades del input, a ver si queda A .
 - “Si A es NP-hard y es subtask de B , entonces B también es NP-hard”

Formas de pensar reducciones (cont.)

- Reemplazo local: $[SAT \rightarrow 3\text{-SAT}]$, $[MCI \rightarrow \text{Max Clique}]$
 - Hacemos un reemplazo “directo” de cada elemento de A , por algunos elementos de B .
 - Luego demostramos que esos elementos de B “se comportan como el elemento de A ” reemplazado.
 - No hay una interacción compleja a nivel de todo el problema entre los elementos.
- Diseño de componentes (Gadgets) $[3\text{-SAT} \rightarrow MCI]$
 - En general, estas son las demostraciones más complicadas e ingeniosas
 - Hay dos tipos de componentes generalmente: los que permiten “tomar decisiones”, y los que permiten “verificar condiciones” (testers).
 - Los componentes se enganchan de tal manera que las decisiones “se comunican” hasta los testers.
 - Todos los componentes interactúan entre sí, tanto por conexiones directas, como por restricciones globales del problema.

Ejemplos de diseño de componentes: 3DM

- 3D Matching es NP-hard (a diferencia de 2D matching)
- Existencia de Ciclo/Camino Hamiltoniano es NP-hard

Escape from Jail

- ¿Hicieron Escape From Jail?
<https://www.spoj.com/problems/ESJAIL/>
- ¿Qué pasa si queremos minimizar la cantidad de movimientos?

El Garey and Johnson, excelente, clásico:

https://en.wikipedia.org/wiki/Computers_and_Intractability